

2026-05-21 - Using Zephyr Drivers and Device Binding

Goal

Learn how Zephyr drivers are accessed in code and how to bind to hardware devices.

What you will learn

- The difference between `device_get_binding()` and `DEVICE_DT_GET()`
- How to use Zephyr device driver APIs
- How to check device readiness
- How to work with common peripherals like GPIO, I2C, and UART

Overview

Zephyr provides a device model where peripherals are exposed as `struct device` objects. Your application can request these devices by name or via Device Tree macros.

Driver access patterns

- `device_get_binding("UART_0")` for named devices
- `DEVICE_DT_GET(DT_NODELABEL(uart0))` for Device Tree-based access
- `device_is_ready(dev)` to verify the device

Example workflow

1. Enable the peripheral in `prj.conf` and overlay
2. Get the device binding in `main()`
3. Use the driver API for operations like GPIO set/reset or I2C read/write

Why this matters

Correct device binding ensures your code works with the actual hardware and is portable across boards.

Practice tasks

1. Bind to a GPIO device and toggle an LED.
2. Bind to a UART device and send a test string.
3. Explore a sample driver API such as sensor, PWM, or I2C.

Next topic

Tomorrow, we will cover debugging and testing Zephyr applications.